Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №4

з дисципліни

«Об’єктно-орієнтоване програмування»

на тему

«Вдосконалення структури коду графічного редактора об’єктів»

Виконав:
студент групи ІМ-51
Демченко Олександр Стефанович
Номер у списку - 10

Перевірив:
Рекечинський Д.О.

Київ 2026

Мета: Мета роботи – отримати вміння та навички проектування класів, виконавши модернізацію коду графічного редактора в об’єктно-орієнтованому стилі для забезпечення зручного додавання нових типів об'єктів.

Завдання

1. Створити у середовищі розробки проект з ім’ям Lab4.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. Налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт

Варіант завдання

Ж = 10

Перелік фігур у меню «Об'єкти»: Крапка, Лінія, Прямокутник, Еліпс, Лінія з кружечками, Куб.

Структура даних: Статичний масив `pcshape` на 111 елементів із контроль переповнення.

«Гумовий» слід: Пунктирна чорна лінія під час перетягування миші.

Вихідний текст головного файлу

MainFrame.java

```
package Lab4;

import javax.swing.*;
import java.awt.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;

public class MainFrame extends JFrame {
  public static final int MODE_POINT = 1;
   public static final int MODE_LINE = 2;
   public static final int MODE_RECTANGLE = 3;
   public static final int MODE_ELLIPSE = 4;
   public static final int MODE_LINE_OO = 5;
   public static final int MODE_CUBE = 6;

   private int currentMode = MODE_POINT;
```

```
//Динамічний об'єкт класу MyEditor
private MyEditor editor;

//Menu
private JMenuBar menuBar;
private JRadioButtonMenuItem itemPoint, itemLine, itemRectangle, itemEllipse, itemLineOO, itemCube;
private ButtonGroup menuGroup;

//Toolbar
private JToolBar toolBar;
private JToggleButton btnPoint, btnLine, btnRectangle, btnEllipse, btnLineOO, btnCube;
private ButtonGroup toolGroup;

private CanvasPanel canvasPanel;

public MainFrame() {
   setSize(800, 600);
   setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
   setLocationRelativeTo(null);
   setLayout(new BorderLayout());

   //Динамічне створення екземпляра MyEditor
   editor = new MyEditor();

   initMenu();
   initToolBar();

   canvasPanel = new CanvasPanel();
   add(canvasPanel, BorderLayout.CENTER);

   setMode(MODE_POINT);
}

private void updateTitle() {
   String modeName = switch (currentMode) {
      case MODE_POINT -> "Крапка";
      case MODE_LINE -> "Лінія";
      case MODE_RECTANGLE -> "Прямокутник";
      case MODE_ELLIPSE -> "Еліпс";
      case MODE_LINE_OO -> "Лінія з кружечками";
      case MODE_CUBE -> "Каркас куба";
      default -> "";
   };
   setTitle("Лабораторна робота №4 - [Обрано об'єкт: " + modeName + "]");
}

//Синхронізована зміна режиму
private void setMode(int mode) {
   this.currentMode = mode;
```

```
        updateTitle();

        //Синхронізація кнопок Toolbar
        switch (mode) {
            case MODE_POINT -> { btnPoint.setSelected(true); itemPoint.setSelected(true); }
            case MODE_LINE -> { btnLine.setSelected(true); itemLine.setSelected(true); }
            case MODE_RECTANGLE -> { btnRectangle.setSelected(true); itemRectangle.setSelected(true); }
            case MODE_ELLIPSE -> { btnEllipse.setSelected(true); itemEllipse.setSelected(true); }
            case MODE_LINE_OO -> { btnLineOO.setSelected(true); itemLineOO.setSelected(true); }
            case MODE_CUBE -> { btnCube.setSelected(true); itemCube.setSelected(true); }
        }
    }

    private void initMenu() {
        menuBar = new JMenuBar();

        JMenu menuFile = new JMenu("Файл");
        JMenuItem itemExit = new JMenuItem("Вихід");
        itemExit.addActionListener(e -> System.exit(0));
        menuFile.add(itemExit);

        JMenu menuObjects = new JMenu("Об'єкти");
        menuGroup = new ButtonGroup();

        itemPoint = new JRadioButtonMenuItem("Крапка");
        itemLine = new JRadioButtonMenuItem("Лінія");
        itemRectangle = new JRadioButtonMenuItem("Прямокутник");
        itemEllipse = new JRadioButtonMenuItem("Еліпс");
        itemLineOO = new JRadioButtonMenuItem("Лінія з кружечками");
        itemCube = new JRadioButtonMenuItem("Каркас куба");

        itemPoint.addActionListener(e -> setMode(MODE_POINT));
        itemLine.addActionListener(e -> setMode(MODE_LINE));
        itemRectangle.addActionListener(e -> setMode(MODE_RECTANGLE));
        itemEllipse.addActionListener(e -> setMode(MODE_ELLIPSE));
        itemLineOO.addActionListener(e -> setMode(MODE_LINE_OO));
        itemCube.addActionListener(e -> setMode(MODE_CUBE));

        JRadioButtonMenuItem[] items = {itemPoint, itemLine, itemRectangle, itemEllipse, itemLineOO,
itemCube};
        for (JRadioButtonMenuItem item : items) {
            menuGroup.add(item);
            menuObjects.add(item);
        }

        JMenu menuHelp = new JMenu("Довідка");
        JMenuItem itemAbout = new JMenuItem("Про програму");
        itemAbout.addActionListener(e -> JOptionPane.showMessageDialog(this, "Графічний редактор.
Лабораторна робота №4"));
        menuHelp.add(itemAbout);
```

```
        //Порядок пунктів меню за умовою: Файл | Об'єкти | Довідка
        menuBar.add(menuFile);
        menuBar.add(menuObjects);
        menuBar.add(menuHelp);

        setJMenuBar(menuBar);
    }

    //Допоміжний метод для автоматичного масштабування PNG під розмір кнопки
    private ImageIcon loadScaledIcon(String path, int width, int height) {
        java.net.URL imgURL = getClass().getResource(path);
        if (imgURL != null) {
            ImageIcon originalIcon = new ImageIcon(imgURL);
            Image scaledImg = originalIcon.getImage().getScaledInstance(width, height,
Image.SCALE_SMOOTH);
            return new ImageIcon(scaledImg);
        }
        return null;
    }

    private void initToolBar() {
        toolBar = new JToolBar();
        toolBar.setFloatable(false);
        toolGroup = new ButtonGroup();

        btnPoint = new JToggleButton(loadScaledIcon("/Lab3/icons/point.png", 20, 20));
        btnLine = new JToggleButton(loadScaledIcon("/Lab3/icons/line.png", 20, 20));
        btnRectangle = new JToggleButton(loadScaledIcon("/Lab3/icons/rect.png", 20, 20));
        btnEllipse = new JToggleButton(loadScaledIcon("/Lab3/icons/ellipse.png", 20, 20));
        btnLineOO = new JToggleButton(loadScaledIcon("/Lab3/icons/LineCircles.jpg", 20, 20));
        btnCube = new JToggleButton(loadScaledIcon("/Lab3/icons/cube.png", 20, 20));

        btnPoint.setToolTipText("Крапка");
        btnLine.setToolTipText("Лінія");
        btnRectangle.setToolTipText("Прямокутник");
        btnEllipse.setToolTipText("Еліпс");
        btnLineOO.setToolTipText("Лінія з кружечками");
        btnCube.setToolTipText("Каркас куба");

        btnPoint.addActionListener(e -> setMode(MODE_POINT));
        btnLine.addActionListener(e -> setMode(MODE_LINE));
        btnRectangle.addActionListener(e -> setMode(MODE_RECTANGLE));
        btnEllipse.addActionListener(e -> setMode(MODE_ELLIPSE));
        btnLineOO.addActionListener(e -> setMode(MODE_LINE_OO));
        btnCube.addActionListener(e -> setMode(MODE_CUBE));

        JToggleButton[] buttons = {btnPoint, btnLine, btnRectangle, btnEllipse, btnLineOO, btnCube};
        for (JToggleButton btn : buttons) {
            toolGroup.add(btn);
```

```
        toolBar.add(btn);
    }

    add(toolBar, BorderLayout.NORTH);
}

private class CanvasPanel extends JPanel {
    private int currentX, currentY;

    public CanvasPanel() {
        setBackground(Color.WHITE);

        MouseAdapter mouseHandler = new MouseAdapter() {
            @Override
            public void mousePressed(MouseEvent e) {
                currentX = e.getX();
                currentY = e.getY();
                editor.startShape(currentMode, currentX, currentY);
            }

            @Override
            public void mouseDragged(MouseEvent e) {
                currentX = e.getX();
                currentY = e.getY();
                editor.dragShape(currentX, currentY);
                repaint();
            }

            @Override
            public void mouseReleased(MouseEvent e) {
                currentX = e.getX();
                currentY = e.getY();
                boolean ok = editor.releaseShape(currentX, currentY);
                if (!ok) {
                    JOptionPane.showMessageDialog(MainFrame.this,
                            "Переповнення масиву об'єктів (111)!",
                            "Помилка", JOptionPane.ERROR_MESSAGE);
                }
                repaint();
            }
        };

        addMouseListener(mouseHandler);
        addMouseMotionListener(mouseHandler);
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        editor.drawAll(g, currentX, currentY);
```

```
    }
  }

  public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new MainFrame().setVisible(true));
  }
}
```

MyEditor.java

```
package Lab4;

import java.awt.Graphics;
import java.awt.event.MouseEvent;

public class MyEditor {
  private static final int MAX_SHAPES = 111;
  private final Shape[] pcshape = new Shape[MAX_SHAPES];
  private int shapeCount = 0;

  private Shape currentShape = null;
  private int startX, startY;
  private boolean isDragging = false;

  public boolean addShape(Shape shape) {
    if (shapeCount < MAX_SHAPES) {
      pcshape[shapeCount++] = shape;
      return true;
    }
    return false;
  }

  public void startShape(int mode, int x, int y) {
    startX = x;
    startY = y;

    currentShape = switch (mode) {
      case MainFrame.MODE_POINT -> new PointShape();
      case MainFrame.MODE_LINE -> new LineShape();
      case MainFrame.MODE_RECTANGLE -> new RectangleShape();
      case MainFrame.MODE_ELLIPSE -> new EllipseShape();
      case MainFrame.MODE_LINE_OO -> new LineCirclesShape();
      case MainFrame.MODE_CUBE -> new CubeShape();
      default -> null;
    };

    if (currentShape != null) {
      currentShape.setPoints(startX, startY, startX, startY);
      isDragging = true;
    }
```

```
    }

    public void dragShape(int x, int y) {
        if (isDragging && currentShape != null) {
            currentShape.setPoints(startX, startY, x, y);
        }
    }

    public boolean releaseShape(int x, int y) {
        if (isDragging && currentShape != null) {
            currentShape.setPoints(startX, startY, x, y);
            boolean added = addShape(currentShape);
            currentShape = null;
            isDragging = false;
            return added;
        }
        return true;
    }

    public void drawAll(Graphics g, int currentX, int currentY) {
        for (int i = 0; i < shapeCount; i++) {
            if (pcshape[i] != null) {
                pcshape[i].show(g);
            }
        }
        if (isDragging && currentShape != null) {
            currentShape.trail(g, currentX, currentY);
        }
    }

    public boolean isDragging() {
        return isDragging;
    }
}
```

Shape.java

```
package Lab4;
import java.awt.Graphics;

public abstract class Shape {
    protected int x1, y1, x2, y2;

    public void setPoints(int x1, int y1, int x2, int y2) {
        this.x1 = x1;
        this.y1 = y1;
        this.x2 = x2;
        this.y2 = y2;
    }
```

```
    public abstract void show(Graphics g);
    public abstract void trail(Graphics g, int currentX, int currentY);
}
```

PointShape.java

```
package Lab4;
import java.awt.Color;
import java.awt.Graphics;

public class PointShape extends Shape {
    @Override
    public void show(Graphics g) {
        g.setColor(Color.BLACK);
        g.fillRect(x1, y1, 2, 2);
    }

    @Override
    public void trail(Graphics g, int currentX, int currentY) {
        g.setColor(Color.BLACK);
        g.fillRect(currentX, currentY, 2, 2);
    }
}
```

LineShape.java

```
package Lab4;
import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;

public class LineShape extends Shape {
    @Override
    public void show(Graphics g) {
        g.setColor(Color.BLACK);
        g.drawLine(x1, y1, x2, y2);
    }

    @Override
    public void trail(Graphics g, int currentX, int currentY) {
        //"Гумовий" слід: пунктирна лінія чорного кольору
        Graphics2D g2d = (Graphics2D) g.create();
        g2d.setColor(Color.BLACK);
        float[] dash = {5.0f, 5.0f};
        g2d.setStroke(new BasicStroke(1.0f, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER, 10.0f, dash,
0.0f));
        g2d.drawLine(x1, y1, currentX, currentY);
        g2d.dispose();
```

```
    }
}
```

RectangleShape.java

```
package Lab4;
import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;

public class RectangleShape extends Shape {
    @Override
    public void show(Graphics g) {
        //Ввід від центру до кута
        int rx = Math.abs(x2 - x1);
        int ry = Math.abs(y2 - y1);
        int x = x1 - rx;
        int y = y1 - ry;
        int width = rx * 2;
        int height = ry * 2;

        //Сіре заповнення
        g.setColor(Color.GRAY);
        g.fillRect(x, y, width, height);

        //Чорний контур
        g.setColor(Color.BLACK);
        g.drawRect(x, y, width, height);
    }

    @Override
    public void trail(Graphics g, int currentX, int currentY) {
        //Пунктирний чорний "гумовий" слід від центру
        Graphics2D g2d = (Graphics2D) g.create();
        g2d.setColor(Color.BLACK);
        float[] dash = {5.0f, 5.0f};
        g2d.setStroke(new BasicStroke(1.0f, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER, 10.0f, dash,
0.0f));

        int rx = Math.abs(currentX - x1);
        int ry = Math.abs(currentY - y1);
        int x = x1 - rx;
        int y = y1 - ry;
        int width = rx * 2;
        int height = ry * 2;

        g2d.drawRect(x, y, width, height);
        g2d.dispose();
    }
```

```
}
```

EllipseShape.java

```
package Lab4;
import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;

public class EllipseShape extends Shape {
    @Override
    public void show(Graphics g) {
        //Ввід по двох протилежних кутах
        int x = Math.min(x1, x2);
        int y = Math.min(y1, y2);
        int width = Math.abs(x2 - x1);
        int height = Math.abs(y2 - y1);

        //Біле заповнення + чорний контур
        g.setColor(Color.WHITE);
        g.fillOval(x, y, width, height);

        g.setColor(Color.BLACK);
        g.drawOval(x, y, width, height);
    }

    @Override
    public void trail(Graphics g, int currentX, int currentY) {
        //Пунктирний чорний "гумовий" слід по двох кутах
        Graphics2D g2d = (Graphics2D) g.create();
        g2d.setColor(Color.BLACK);
        float[] dash = {5.0f, 5.0f};
        g2d.setStroke(new BasicStroke(1.0f, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER, 10.0f, dash,
0.0f));

        int x = Math.min(x1, currentX);
        int y = Math.min(y1, currentY);
        int width = Math.abs(currentX - x1);
        int height = Math.abs(currentY - y1);

        g2d.drawOval(x, y, width, height);
        g2d.dispose();
    }
}
```

LineCirclesShape.java

```
package Lab4;
import java.awt.Graphics;

public class LineCirclesShape extends Shape {
   private final LineShape line = new LineShape();
   private final EllipseShape circle1 = new EllipseShape();
   private final EllipseShape circle2 = new EllipseShape();
   private static final int RADIUS = 6;

   @Override
   public void setPoints(int x1, int y1, int x2, int y2) {
      super.setPoints(x1, y1, x2, y2);
      line.setPoints(x1, y1, x2, y2);
      circle1.setPoints(x1 - RADIUS, y1 - RADIUS, x1 + RADIUS, y1 + RADIUS);
      circle2.setPoints(x2 - RADIUS, y2 - RADIUS, x2 + RADIUS, y2 + RADIUS);
   }

   @Override
   public void show(Graphics g) {
      line.show(g);
      circle1.show(g);
      circle2.show(g);
   }

   @Override
   public void trail(Graphics g, int currentX, int currentY) {
      line.trail(g, currentX, currentY);
   }
}
```

CubeShape.java

```
package Lab4;

import java.awt.Graphics;

public class CubeShape extends Shape {
   private final RectangleShape rectFront = new RectangleShape();
   private final RectangleShape rectBack = new RectangleShape();
   private final LineShape line1 = new LineShape();
   private final LineShape line2 = new LineShape();
   private final LineShape line3 = new LineShape();
   private final LineShape line4 = new LineShape();

   @Override
   public void setPoints(int x1, int y1, int x2, int y2) {
      super.setPoints(x1, y1, x2, y2);

      int totalRx = Math.abs(x2 - x1);
      int totalRy = Math.abs(y2 - y1);
```

```
    //Радіуси (півширина та піввисота) для кожного з двох прямокутників
    int frx = totalRx / 2;
    int fry = totalRy / 2;

    //Зсув між передньою та задньою гранню
    int dx = frx;
    int dy = fry;

    //Центр переднього прямокутника
    int fcx = x1 - dx / 2;
    int fcy = y1 + dy / 2;

    //Центр заднього прямокутника
    int bcx = x1 + dx / 2;
    int bcy = y1 - dy / 2;

    //Задаємо центри та радіуси для прямокутників
    rectFront.setPoints(fcx, fcy, fcx + frx, fcy + fry);
    rectBack.setPoints(bcx, bcy, bcx + frx, bcy + fry);

    //4 реальні кути переднього прямокутника
    int fTLx = fcx - frx, fTLy = fcy - fry; // Верхній лівий
    int fTRx = fcx + frx, fTRy = fcy - fry; // Верхній правий
    int fBRx = fcx + frx, fBRy = fcy + fry; // Нижній правий
    int fBLx = fcx - frx, fBLy = fcy + fry; // Нижній лівий

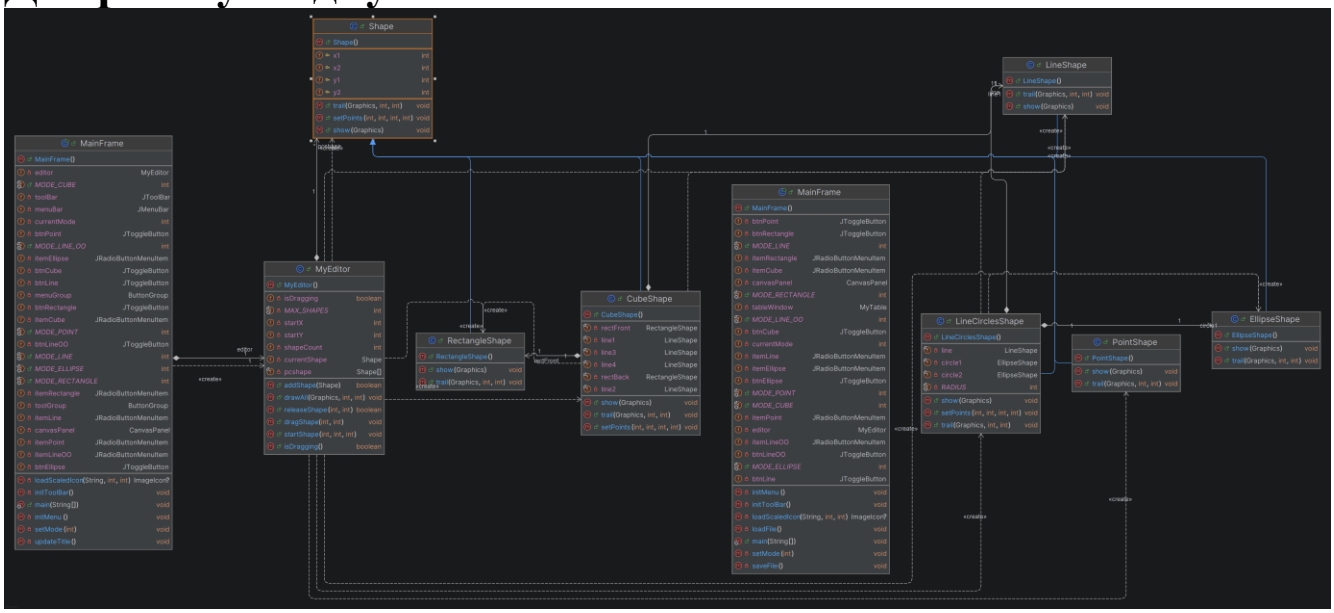
    //4 реальні кути заднього прямокутника
    int bTLx = bcx - frx, bTLy = bcy - fry;
    int bTRx = bcx + frx, bTRy = bcy - fry;
    int bBRx = bcx + frx, bBRy = bcy + fry;
    int bBLx = bcx - frx, bBLy = bcy + fry;

    //4 з'єднувальні лінії точно по кутах прямокутників
    line1.setPoints(fTLx, fTLy, bTLx, bTLy); // Верхній лівий
    line2.setPoints(fTRx, fTRy, bTRx, bTRy); // Верхній правий
    line3.setPoints(fBRx, fBRy, bBRx, bBRy); // Нижній правий
    line4.setPoints(fBLx, fBLy, bBLx, bBLy); // Нижній лівий
}

@Override
public void show(Graphics g) {
    rectBack.show(g);
    rectFront.show(g);
    line1.show(g);
    line2.show(g);
    line3.show(g);
    line4.show(g);
}
```

```
    @Override
    public void trail(Graphics g, int currentX, int currentY) {
        setPoints(x1, y1, currentX, currentY);
        show(g);
    }
}
```

Діаграма успадкування класів



Для кращого перегляду діаграму розміщено на github -> [Diagram]

Ілюстрації роботи програми

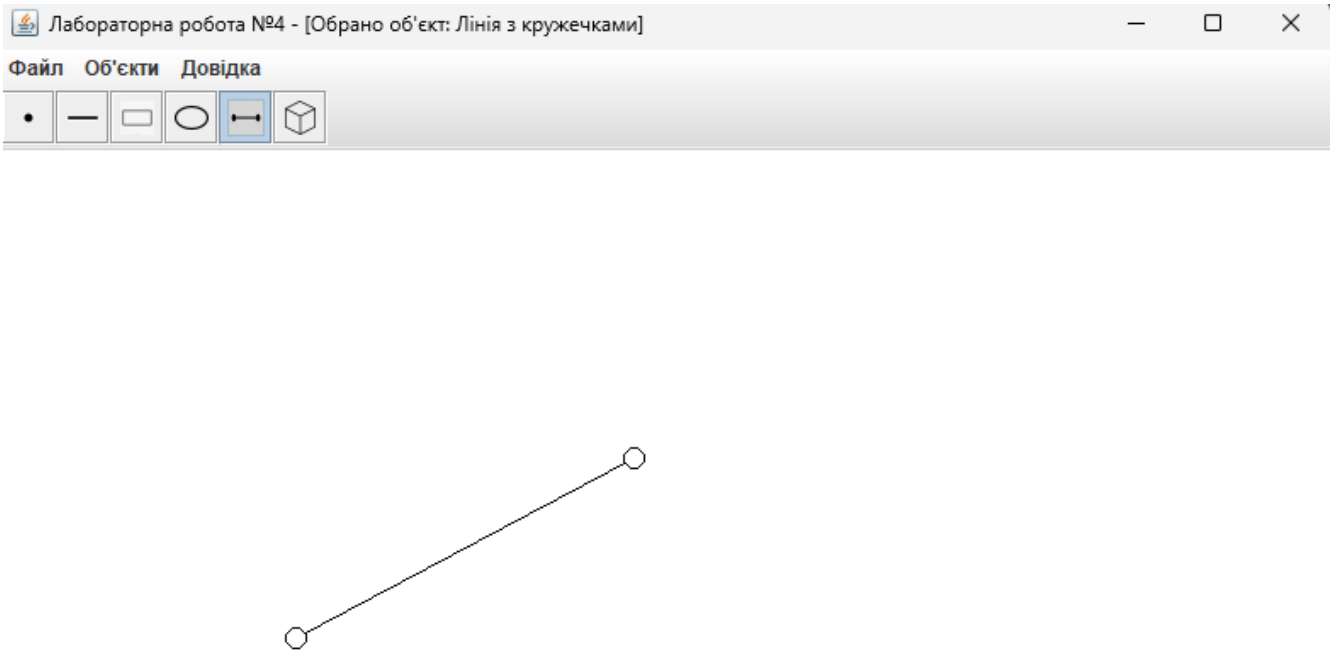


Рис.1 - нова фігура “лінія з кружечками”

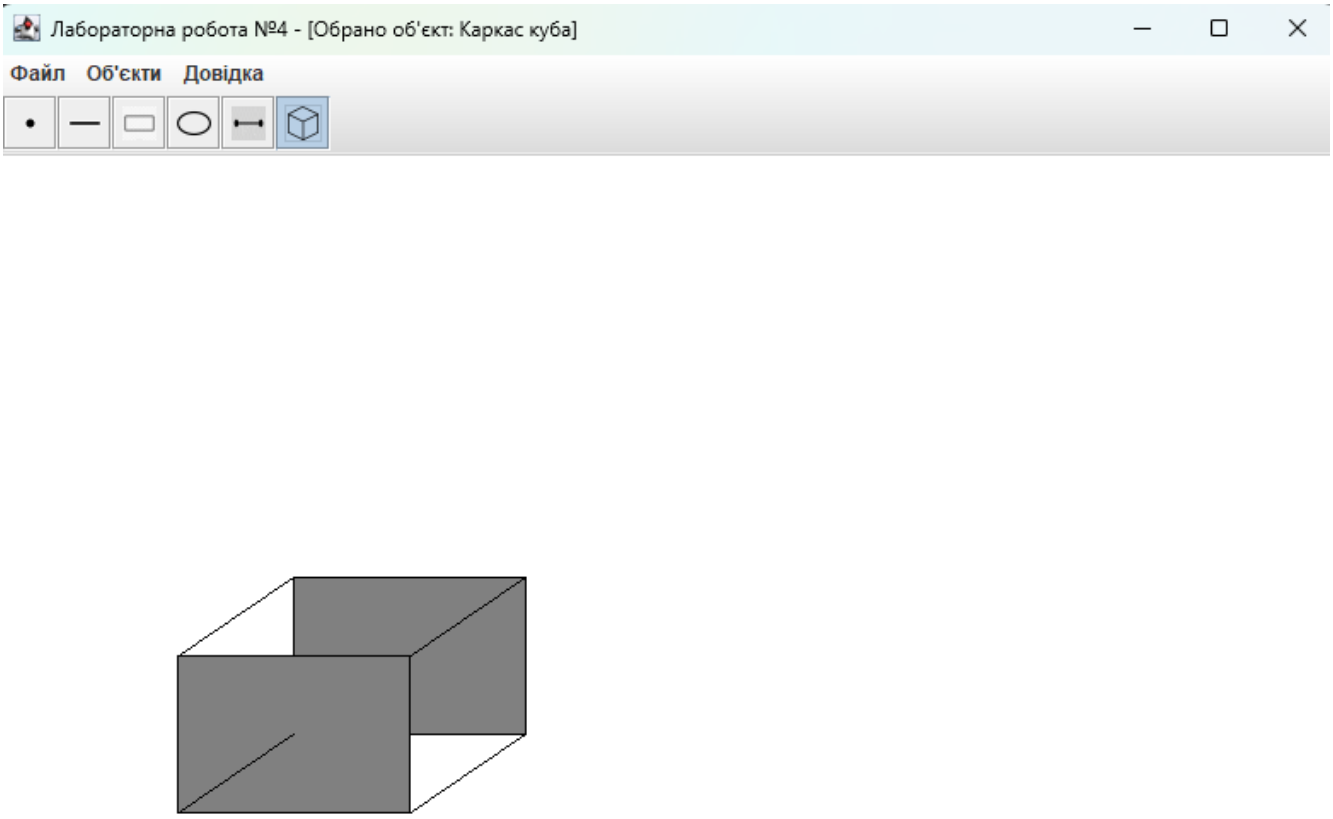


Рис.2 - нова фігура “Куб”

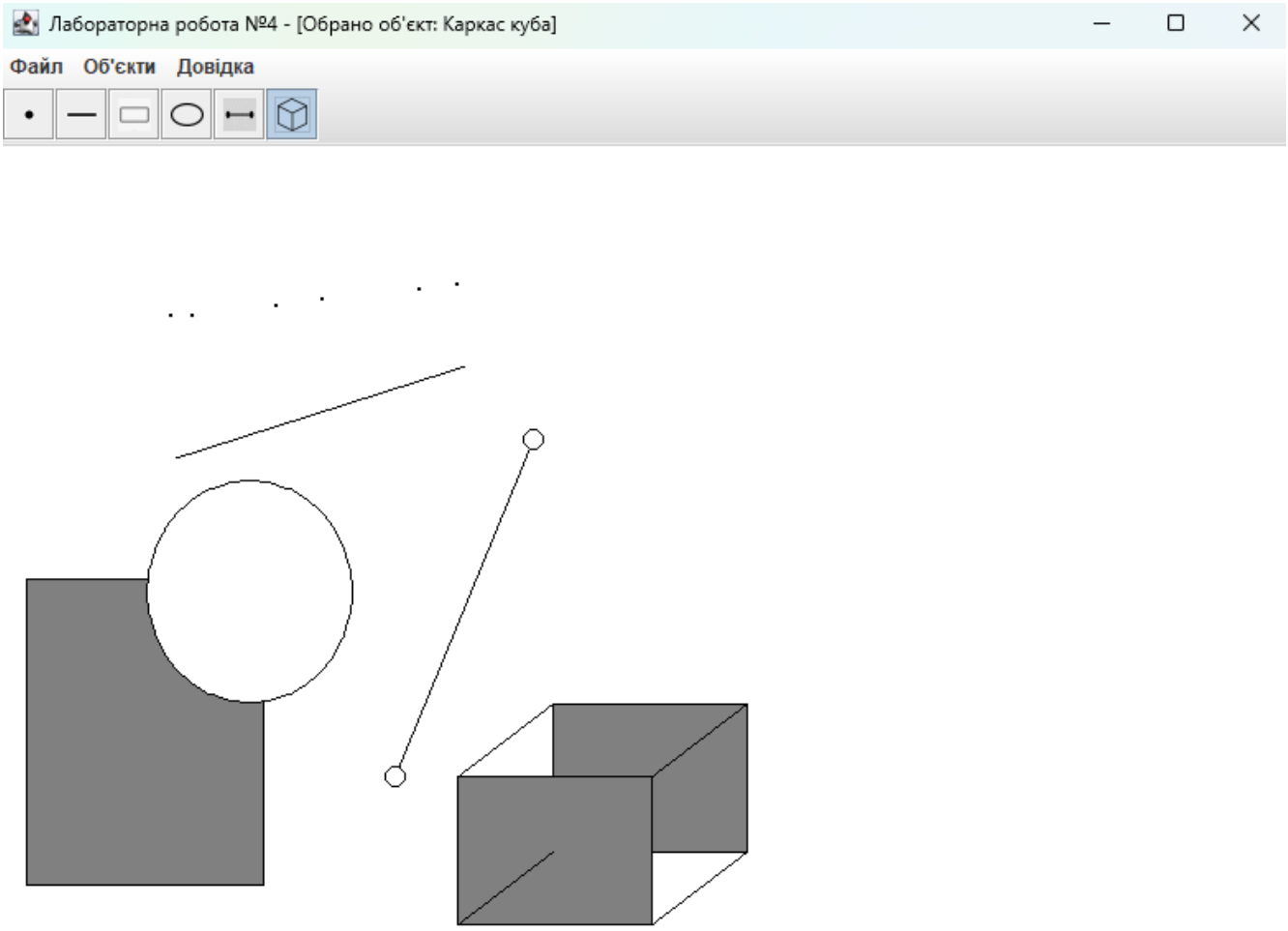


Рис.3 - демонстрація фігур з попередніх лабораторних робіт

Висновок

У ході виконання лабораторної роботи №4 було вдосконалено архітектуру графічного редактора шляхом винесення логіки керування малюванням у динамічний об'єкт MyEditor, а також розширено функціонал двома новими фігурами — лінією з кружечками та каркасом куба. Оскільки Java не підтримує множинне успадкування класів, нові фігури було реалізовано за допомогою композиції, що дозволило повторно використати готові методи show() існуючих класів LineShape, RectangleShape та EllipseShape без зміни їхнього коду. У результаті було розширено панель інструментів JToolBar до 6 фігур з підказками та іконками, що підтвердило гнучкість розробленої об'єктно-орієнтованої структури.